



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



GUÍA PRÁCTICA GRUPAL SEMANA 14

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad II: Desarrollo Web Fullstack

Tema: Microservicios con Django, Arquitectura de Contenedores (Docker), Orquestación Básica (Kubernetes), Comunicación Inter-Servicios, Seguridad, Monitoreo y Escalado

Modalidad: Presencial / Laboratorio

Fecha: 09.07.2026

Duración: 90 minutos

INTEGRANTES:

- BARJA ORTIZ, ERICK
- NAVARRO SERVA, LESLY
- SULLUCHUCO VILCAPOMA, ANYELO
- TORIBIO ANSELMO, DAVID ANGEL
- YAURI TORRES, BENJAMIN

Consigna Técnica

Formen equipos 6 estudiantes. Deben implementar un **Ecosistema de Microservicios (Auth + Catálogo)** que cumpla:

1. Arquitectura & Aislamiento: Dos servicios Django independientes ([authservice](#), [catalog-service](#)). DB PostgreSQL separada por servicio. Variables de entorno segregadas.
2. Contenedorización & Orquestación: [Dockerfile](#) optimizado (multi-stage, non-root), [docker-compose.yml](#) para desarrollo, manifests K8s ([Deployment](#), [Service](#), [Ingress](#), [ConfigMap](#), [Secret](#)). Desplegado en Minikube/k3s.
3. Comunicación & Seguridad: [auth-service](#) emite JWT. [catalog-service](#) valida token en headers. Cliente inter-servicio con [requests](#) + retries + timeout. [XRequest-ID](#) para tracing.
4. Monitoreo, Health & Escalado: Endpoint [/health/](#) con checks de DB y dependencias. [readinessProbe](#)/[livenessProbe](#) configurados. `kubectl scale -replicas=3` validado. Métricas básicas expuestas.



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



5. Consumo & Validación: Script cliente o colección Postman que autentica, obtiene token, consulta catálogo, maneja [401/429/503](#). Documentación de arquitectura y decisiones técnicas.

Roles Técnicos

Rol	Responsabilidad Detallada
Arquitecto de Microservicios & DB	Define bounded contexts, estructura de proyectos, aislamiento de DB, migraciones independientes, validación de consistencia eventual vs transaccional.
Ingeniero de Contenedores & K8s	Escribe Dockerfile seguro, docker-compose.yml, manifests K8s (Deployment, Service, Ingress, ConfigMap, Secret), aplica con kubectl, valida rollout.
Especialista en Comunicación & Seguridad	Implementa JWT, validación de tokens, cliente HTTP interservicio, retries, timeouts, X-Request-ID, gestión de secretos, políticas de red básicas.
QA de Observabilidad & Escalado	Configura /health/, readiness/livenessProbes, prueba escalado horizontal, valida métricas, ejecuta pruebas de carga leves, documenta en README.md.

Entregable

- Link del Repositorio GitHub o ZIP con: [auth-service/](#), [catalog-service/](#), [k8s/](#), [docker-compose.yml](#), [.env.example](#), [README.md](#)
- [README.md](#) debe incluir:
 - Diagrama de arquitectura (servicios, DBs, red, ingress, cliente externo)
 - Matriz de comunicación (métodos, endpoints, headers, timeouts, retry policy)
 - Capturas: [kubectl get pods/svc/ingress](#), logs de contenedores, escalado en acción, respuesta JWT válida
 - Reflexión técnica: [¿Cuándo justifica un monolito vs microservicios? ¿Qué falla primero si se omite readinessProbe? ¿Cómo escala K8s sin downtime?](#)
- Capturas de pantalla de todo el procedimiento de la implementación del **Ecosistema de Microservicios (Auth + Catálogo)**

Rúbrica de Evaluación (Laboratorio Semana 14)

Criterio	Peso	Indicadores Técnicos
Arquitectura aislada + DB per Service + estructura clara	20%	Servicios independientes, esquemas separados, migraciones propias, sin código compartido



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Contenedorización segura + K8s manifests funcionales	25%	Dockerfile multi-stage/non-root, docker-compose operativo, Deployment/Service/Ingress aplicados, probes configurados
Comunicación inter-servicio + JWT + retries/timeouts	20%	Token emitido/validado, headers propagados, cliente con backoff/retry, X-Request-ID implementado
Monitoreo, health checks + escalado validado	20%	/health/ funcional, probes correctos, kubectl scale sin error, métricas básicas expuestas
Documentación, reflexión técnica y trabajo colaborativo	15%	README completo, diagrama arquitectónico, matriz de comunicación, reflexiones fundamentadas, entregable estructurado

CÓDIGO BASE DE LA GUÍA PRÁCTICA (Refactorizar) auth-service/Dockerfile

dockerfile

```
1 FROM python:3.11-slim AS builder
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5
6 FROM python:3.11-slim
7 WORKDIR /app
8 COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/site-packages
9 COPY . .
10 RUN useradd -m appuser && chown -R appuser:appuser /app
11 USER appuser
12 EXPOSE 8000
13 CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "auth_service.wsgi:application"]
```

docker -compose.yml



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



yaml

```
1 version: '3.9'
2 services:
3   auth-db:
4     image: postgres:15-alpine
5     environment:
6       POSTGRES_DB: authdb
7       POSTGRES_USER: authuser
8       POSTGRES_PASSWORD: authpass
9     volumes: [auth-data:/var/lib/postgresql/data]
10
11   catalog-db:
12     image: postgres:15-alpine
13     environment:
14       POSTGRES_DB: catalogdb
15       POSTGRES_USER: cataloguser
16       POSTGRES_PASSWORD: catalogpass
17     volumes: [catalog-data:/var/lib/postgresql/data]
18
19   auth-service:
20     build: ./auth-service
21     ports: ["8001:8000"]
22     environment:
23       DATABASE_URL: postgres://authuser:authpass@auth-db:5432/authdb
24       JWT_SECRET: dev-secret-change-in-prod
25     depends_on: [auth-db]
26
27   catalog-service:
28     build: ./catalog-service
29     ports: ["8002:8000"]
30     environment:
31       DATABASE_URL: postgres://cataloguser:catalogpass@catalog-db:5432/catalogdb
32       AUTH_SERVICE_URL: http://auth-service:8000
33     depends_on: [catalog-db, auth-service]
34
35 volumes:
36   auth-data:
37   catalog-data:
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



k8s/deployment-auth.yaml

yaml

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: auth-deploy
5    labels: {app: auth}
6  spec:
7    replicas: 2
8    selector: {matchLabels: {app: auth}}
9    template:
10     metadata: {labels: {app: auth}}
11     spec:
12       containers:
13       - name: auth
14         image: auth-service:latest
15         imagePullPolicy: IfNotPresent
16         ports: [{containerPort: 8000}]
17         env:
18         - name: DATABASE_URL
19           valueFrom: {secretKeyRef: {name: auth-db-secret, key: url}}
20         - name: JWT_SECRET
21           valueFrom: {secretKeyRef: {name: auth-db-secret, key: jwt}}
22       readinessProbe:
23         httpGet: {path: /health/, port: 8000}
24         initialDelaySeconds: 10
25         periodSeconds: 5
26       livenessProbe:
27         httpGet: {path: /health/, port: 8000}
28         initialDelaySeconds: 15
29         periodSeconds: 10
30       resources:
31         requests: {cpu: 100m, memory: 128Mi}
32         limits: {cpu: 250m, memory: 256Mi}
```



auth-service/views.py (JWT + Health)

python

```
1 import jwt, datetime
2 from django.http import JsonResponse
3 from django.views.decorators.csrf import csrf_exempt
4 from rest_framework.decorators import api_view
5 from rest_framework.response import Response
6
7 @csrf_exempt
8 @api_view(['POST'])
9 def login(request):
10     # TODO: Validar credenciales, generar JWT con exp y scope
11     payload = {'user_id': 1, 'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)}
12     token = jwt.encode(payload, request.META.get('JWT_SECRET', 'dev'), algorithm='HS256')
13     return Response({'token': token})
14
15 @api_view(['GET'])
16 def health(request):
17     # TODO: Validar conexión a DB y dependencias
18     import psycopg2
19     try:
20         conn = psycopg2.connect(request.META['DATABASE_URL'])
21         conn.close()
22         return Response({'status': 'healthy', 'db': 'connected'})
23     except Exception as e:
24         return Response({'status': 'unhealthy', 'db': str(e)}, status=503)
```

catalog-service/utils/client.py (Inter-Service Call)

python

```
1 import requests, os
2 from requests.adapters import HTTPAdapter
3 from urllib3.util.retry import Retry
4
5 session = requests.Session()
6 retry = Retry(total=3, backoff_factor=0.5, status_forcelist=[500, 502, 503, 504])
7 session.mount('http://', HTTPAdapter(max_retries=retry))
8
9 def call_auth_service(endpoint: str, headers: dict = None):
10     url = os.getenv('AUTH_SERVICE_URL', 'http://auth-service:8000') + endpoint
11     headers = headers or {}
12     headers.setdefault('X-Request-ID', os.urandom(16).hex())
13     try:
14         return session.get(url, headers=headers, timeout=3)
15     except requests.RequestException as e:
16         return {'error': str(e), 'status': 503}
```




UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



catalog-service/middleware.py (JWT Validation)

python

```
1 import jwt, os
2 from django.http import JsonResponse
3 from django.utils.deprecation import MiddlewareMixin
4
5 class JWTAuthMiddleware(MiddlewareMixin):
6     def process_request(self, request):
7         if request.path.startswith('/admin/') or request.path == '/health/':
8             return None
9         auth_header = request.headers.get('Authorization')
10        if not auth_header or not auth_header.startswith('Bearer '):
11            return JsonResponse({'error': 'Token missing'}, status=401)
12        token = auth_header.split(' ')[1]
13        try:
14            payload = jwt.decode(token, os.getenv('JWT_SECRET', 'dev'), algorithms=['HS256'])
15            request.user_id = payload.get('user_id')
16        except jwt.ExpiredSignatureError:
17            return JsonResponse({'error': 'Token expired'}, status=401)
18        except jwt.InvalidTokenError:
19            return JsonResponse({'error': 'Invalid token'}, status=403)
```

NOTAS TÉCNICAS CLAVE

Concepto	Explicación Técnica Extendida
Microservicios vs Monolito	Microservicios favorecen despliegue independiente, escalado granular y tecnología heterogénea, pero añaden complejidad en redes, consistencia, observabilidad y CI/CD. Usar solo cuando el dominio justifica bounded contexts claros y equipos independientes.
Database-perService	Cada servicio posee su esquema y conexión. Evita acoplamiento implícito y permite migraciones independientes. La consistencia entre servicios se logra vía eventos o APIs idempotentes, no joins cruzados.
Comunicación Síncrona vs Asíncrona	REST/gRPC para consultas/respuestas inmediatas. Colas (RabbitMQ/Kafka) para fire-and-forget, procesamiento batch o compensación de transacciones. Implementar retries con backoff exponencial, idempotency keys y circuit breakers.
Contenedores & Inmutabilidad	Docker empaqueta código + dependencias + config en imagen inmutable. Multi-stage reduce tamaño. Non-root user previene escalation. readinessProbe indica cuándo aceptar tráfico; livenessProbe reinicia si hay deadlock.
Kubernetes Básico	Deployment gestiona replicas y rollout. Service expuesta cluster-IP con DNS interno. Ingress rutea tráfico externo con TLS/paths. ConfigMap/Secret inyectan config sin rebuild. Declarativo: aplica, no imperativo.



DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS

Seguridad en Microservicios	JWT para stateless auth, validado en cada servicio o API Gateway. Secrets en K8s Secret (base64 + RBAC) o Vault. NetworkPolicies limitan tráfico entre namespaces. TLS mTLS opcional para service mesh.
Observabilidad & Escalado	Logs centralizados (EFK/Loki), métricas (Prometheus), traces (Jaeger). HPA escala pods según CPU/mem o custom metrics. Graceful shutdown (SIGTERM → drain connections → exit). Rollout sin downtime con maxSurge/maxUnavailable.

DESARROLLO DE LA PRÁCTICA:

- Link del Repositorio GitHub o ZIP con:

<https://github.com/benja1004/semana14.git>

- **README.md** debe incluir:
 - Diagrama de arquitectura (servicios, DBs, red, ingress, cliente externo)

```
## 1. Diagrama de Arquitectura
A continuación se detalla la arquitectura lógica de comunicación y red del ecosistema, desde el cliente externo hasta la persistencia separada de bases de datos.

```mermaid
graph TD
 Client[Cliente Externo / Postman] -->|HTTP Requests| Ingress[Kubernetes Ingress Controller]

 subgraph Kubernetes Cluster
 Ingress -->|Route /login/| AuthSvc[Service: auth-service - Port 8000]
 Ingress -->|Route /catalog/| CatSvc[Service: catalog-service - Port 8000]

 AuthSvc -->|Selector| AuthPods[Pods: auth-deploy - 2 Replicas]
 CatSvc -->|Selector| CatPods[Pods: catalog-deploy - 2 Replicas]

 CatPods -->|Inter-service Resilient HTTP Session + Retries| AuthSvc

 AuthPods -->|DB Connection| AuthDB[(PostgreSQL: authdb)]
 CatPods -->|DB Connection| CatDB[(PostgreSQL: catalogdb)]
 end
end
```

- Matriz de comunicación (métodos, endpoints, headers, timeouts, retry policy)

```
2. Matriz de Comunicación de Microservicios

| Servicio Origen | Servicio Destino | Endpoint / Recurso | Método | Cabeceras Requeridas | Timeout | |
|---|---|---|---|---|---|---|
| **Cliente Externo** | `auth-service` | `/login/` | `POST` | `Content-Type: application/json` | - | - |
| **Cliente Externo** | `catalog-service` | `/catalog/` | `GET` | `Authorization: Bearer <JWT>` | - | - |
| **catalog-service** | `auth-service` | `/health/` | `GET` | `X-Request-ID`, `Authorization` | 3.0s |
| `HTTPAdapter` + 3 Reintentos con Backoff Exponencial (Factor: 0.5) en códigos 500, 502, 503, 504 |
| **Kubernetes (Probes)** | `auth-service` | `/health/` | `GET` | - | - | Readiness (10s delay, 5s period) / Liveness (15s delay, 10s period) |
| **Kubernetes (Probes)** | `catalog-service` | `/health/` | `GET` | - | - | Readiness (10s delay, 5s period) / Liveness (15s delay, 10s period) |
```





- Capturas: `kubectl get pods/svc/ingress`, logs de contenedores, escalado en acción, respuesta JWT válida

```
> Y deployment-auth.yaml
spec:
 template:
 metadata:
 labels:
 app: auth
 spec:
 containers:
 - name: auth
 image: auth-service:latest
 imagePullPolicy: IfNotPresent
 ports:
 - containerPort: 8000
 env:
 - name: DATABASE_URL
 valueFrom:
 secretKeyRef:
 name: auth-db-secret
 key: url
 - name: JWT_SECRET
 valueFrom:
 secretKeyRef:
 name: auth-db-secret
 key: jwt
 - name: DJANGO_DEBUG
 valueFrom:
 configMapKeyRef:
 name: app-config
 key: DJANGO_DEBUG
 readinessProbe:
```

```
spec:
 template:
 spec:
 containers:
 - name: catalog
 env:
 - name: DATABASE_URL
 valueFrom:
 secretKeyRef:
 name: catalog-db-secret
 key: url
 - name: JWT_SECRET
 valueFrom:
 secretKeyRef:
 name: catalog-db-secret
 key: jwt
 - name: AUTH_SERVICE_URL
 valueFrom:
 configMapKeyRef:
 name: app-config
 key: AUTH_SERVICE_URL
 - name: DJANGO_DEBUG
 valueFrom:
 configMapKeyRef:
 name: app-config
 key: DJANGO_DEBUG
 readinessProbe:
 httpGet:
```

ms 1 Output Debug Console Terminal Ports



```
Y service-auth.yaml
apiVersion: v1
kind: Service
metadata:
 name: auth-service
 labels:
 app: auth
spec:
 type: ClusterIP
 ports:
 - port: 8000
 targetPort: 8000
 protocol: TCP
 name: http
 selector:
 app: auth
```

```
Y service-catalog.yaml
apiVersion: v1
kind: Service
metadata:
 name: catalog-service
 labels:
 app: catalog
spec:
 type: ClusterIP
 ports:
 - port: 8000
 targetPort: 8000
 protocol: TCP
 name: http
 selector:
 app: catalog
```



## UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ FACULTAD DE INGENIERÍA DE SISTEMAS



```
Y ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: microservices-ingress
 annotations:
 # Anotación estándar para habilitar CORS si el cliente web externo lo requiere
 nginx.ingress.kubernetes.io/enable-cors: "true"
spec:
 rules:
 - http:
 paths:
 # Ruta pública del endpoint de login (/login/) mapeada al auth-service
 - path: /login/
 pathType: Prefix
 backend:
 service:
 name: auth-service
 port:
 number: 8000
 # Ruta pública del catálogo de productos (/catalog/) mapeada al catalog-service
 - path: /catalog/
 pathType: Prefix
 backend:
 service:
 name: catalog-service
 port:
 number: 8000
```

```
Y configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
 name: app-config
 labels:
 app: microservices
data:
 DJANGO_DEBUG: "False"
 AUTH_SERVICE_URL: "http://auth-service:8000"
```

```
Y secrets.yaml
apiVersion: v1
kind: Secret
metadata:
 name: auth-db-secret
 labels:
 app: auth
type: Opaque
data:
 # postgres://authuser:authpass@auth-db:5432/authdb
 url: cG9zdGdyZXM6Ly9hdXRodXNlcjphdXRocGFzc0BhdXRoLWRiOjU0MzIyYXV0aGRi
 # dev-secret-change-in-prod
 jwt: ZGV2LXNlY3JldC1jaGFuZ2UtaW4tcHJvZA==

apiVersion: v1
kind: Secret
metadata:
 name: catalog-db-secret
 labels:
 app: catalog
type: Opaque
data:
 # postgres://cataloguser:catalogpass@catalog-db:5432/catalogdb
 url: cG9zdGdyZXM6Ly9jYXRhbG9ndXNlcjpwYXRhbG9ncGFzc0BjYXRhbG9nLWRiOjU0MzIyYXV0aGRi
```



- Reflexión técnica: ¿Cuándo justifica un monolito vs microservicios? ¿Qué falla primero si se omite readinessProbe? ¿Cómo escala K8s sin downtime?

```
PREGUNTA 1: ¿Cuándo justifica migrar a una arquitectura de microservicios en lugar de mantener un monolito?
Respuesta:
La transición a microservicios se justifica bajo las siguientes circunstancias:
1. **Escalabilidad independiente:** Cuando ciertas partes de la aplicación (como el catálogo) tienen cargas extremadamente altas mientras que otras (como la administración o reportes) son de uso esporádico. Esto permite optimizar el consumo de recursos de Kubernetes asignando réplicas y recursos (CPU/Memoria) por separado.
2. **Ciclos de desarrollo y despliegues independientes:** Cuando hay múltiples sub-equipos de desarrollo trabajando sobre diferentes contextos acotados (*Bounded Contexts*). Los microservicios evitan bloqueos en lanzamientos a producción (*releases*).
3. **Heterogeneidad tecnológica:** Cuando un servicio requiere de una tecnología o lenguaje específico (por ejemplo, procesamiento de datos masivo con Python/Spark y un backend transaccional de alta concurrencia en Go).
4. **Resiliencia de fallos:** Evita que el fallo de un componente no crítico apague la aplicación por completo. En un monolito, una fuga de memoria en un reporte tumba todo el sitio; en microservicios, si el catálogo falla, el servicio de autenticación y pasarela siguen funcionando.
```

```
PREGUNTA 2: ¿Qué falla primero a nivel de Kubernetes si omitimos la configuración del `readinessProbe` durante un despliegue y por qué?
Respuesta:
Si se omite el `readinessProbe`, ocurrirá **pérdida de tráfico o peticiones caídas (502 / 503 Bad Gateway)** durante los despliegues o actualizaciones (*Rollouts*).
- **Por qué sucede:** Por defecto, si no hay un `readinessProbe`, Kubernetes asume que el contenedor está listo para recibir tráfico inmediatamente después de que el proceso principal inicia.
- En aplicaciones de Django que arrancan mediante Unicorn, toma unos segundos cargar los módulos en memoria y conectar con la base de datos PostgreSQL.
- Kubernetes comenzará a enviar peticiones al nuevo Pod antes de que este pueda procesarlas. Las solicitudes que lleguen durante esa ventana fallarán con errores de conexión.
- Con `readinessProbe`, el balanceador de carga del Service no redirigirá tráfico hacia el Pod hasta que este responda HTTP 200 en su endpoint `/health/`.
```

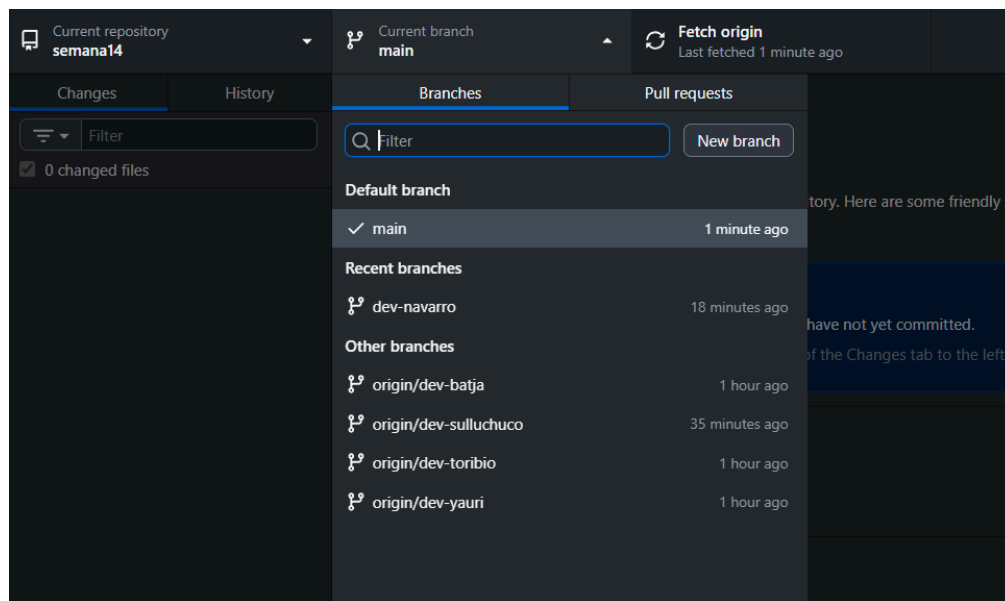
```
PREGUNTA 3: ¿Cómo gestiona Kubernetes el escalado y actualización de pods sin generar interrupción del servicio (Zero-Downtime)?
Respuesta:
Kubernetes logra esto mediante una combinación de la estrategia de actualización **RollingUpdate**, probes de salud, y el ciclo de vida del Pod:
1. **Estrategia RollingUpdate:** Permite definir parámetros de actualización como:
 - `maxSurge`: Cuántos pods adicionales se pueden crear por encima de la cantidad deseada durante la actualización.
 - `maxUnavailable`: Cuántos pods se pueden dar de baja simultáneamente.
2. **Creación y Verificación Progresiva:** Kubernetes crea un nuevo pod con la versión actualizada. El tráfico no se redirige a este pod hasta que el `readinessProbe` devuelva HTTP 200 de forma consistente.
3. **Terminación Controlada (SIGTERM):** Una vez que el pod nuevo está listo, Kubernetes cambia el estado del pod antiguo a `Terminating`, lo remueve del balanceador del Service, y envía una señal `SIGTERM`. Esto le permite al pod antiguo terminar de procesar las conexiones activas (*Graceful Shutdown*) antes de ser destruido.
4. **Reemplazo paulatino:** El ciclo continúa de pod en pod, garantizando que siempre haya capacidad disponible respondiendo activamente peticiones.
```

- Capturas de pantalla de todo el procedimiento de la implementación del **Ecosistema de Microservicios (Auth + Catálogo)**

PASO 1:



# UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ FACULTAD DE INGENIERÍA DE SISTEMAS



## PASO2:

```
import os
from pathlib import Path
from urllib.parse import urlparse

Ruta base del proyecto
BASE_DIR = Path(__file__).resolve().parent.parent

Configuración básica de Django para desarrollo/producción
SECRET_KEY = os.getenv('DJANGO_SECRET_KEY', 'django-insecure-dev-secret-key-semana-14')
DEBUG = os.getenv('DJANGO_DEBUG', 'True') == 'True'

ALLOWED_HOSTS = ['*']

Aplicaciones instaladas
INSTALLED_APPS = [
 'django.contrib.admin',
 'django.contrib.auth',
 'django.contrib.contenttypes',
 'django.contrib.sessions',
 'django.contrib.messages',
 'django.contrib.staticfiles',
 'rest_framework',
 # App de dominio del servicio (modelos + migraciones aislados en authdb)
 'accounts',
]
```

## PASO3:



```
from django.urls import path
from auth_service import views

urlpatterns = [
 path('login/', views.login, name='login'),
 path('health/', views.health, name='health'),
]
```

PASO 4:

```
import datetime
import os
import psycogp2
from django.views.decorators.csrf import csrf_exempt
from rest_framework.decorators import api_view
from rest_framework.response import Response

@csrf_exempt
@api_view(['POST'])
def login(request):
 """
 Endpoint '/login/' (POST)
 Genera un token JWT usando la firma HS256 con una expiración de 1 hora.
 En una aplicación real, se validarían las credenciales del usuario aquí.
 """

 jwt_secret = os.getenv('JWT_SECRET', 'dev-secret-change-in-prod')

 # Tiempo actual compatible con diferentes versiones de Python
 try:
 now = datetime.datetime.now(datetime.timezone.utc)
 except AttributeError:
 now = datetime.datetime.utcnow()

 payload = {
 'user_id': 1,
 'username': 'estudiante_semana14',
 }
```

PASO 5:

```
import os
from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'auth_service.settings')

application = get_wsgi_application()
```

PASO 6:





```
import os
import sys

def main():
 """Run administrative tasks."""
 os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'auth_service.settings')
 try:
 from django.core.management import execute_from_command_line
 except ImportError as exc:
 raise ImportError(
 "Couldn't import Django. Are you sure it's installed and "
 "available on your PYTHONPATH environment variable? Did you "
 "forget to activate a virtual environment?"
) from exc
 execute_from_command_line(sys.argv)

if __name__ == '__main__':
 main()
```

PASO 7:

```
Django>=4.2,<5.0
djangorestframework>=3.14.0
PyJWT>=2.8.0
psycpg2-binary>=2.9.9
gunicorn>=21.2.0
```

PASO 8:

```
name = models.CharField(max_length=200)
description = models.TextField(blank=True)
price = models.DecimalField(max_digits=10, decimal_places=2, default=0)
stock = models.PositiveIntegerField(default=0)
is_available = models.BooleanField(default=True)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)

class Meta:
 db_table = 'catalog_product'
 ordering = ['name']
 verbose_name = 'Producto'
 verbose_name_plural = 'Productos'

def __str__(self):
 return self.name
```



PASO 9:

```
from django.contrib import admin

from catalog.models import Product

@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
 list_display = ('id', 'name', 'price', 'stock', 'is_available', 'created_at')
 search_fields = ('name', 'description')
 list_filter = ('is_available',)
```

PASO 10:

```
from django.urls import path
from catalog_service import views

urlpatterns = [
 path('catalog/', views.catalog_list, name='catalog_list'),
 path('health/', views.health, name='health'),
]
```

PASO 11:

```
@api_view(['GET'])
def health(request):
 """
 Endpoint '/health/' (GET)
 Valida la conexión real a PostgreSQL del servicio de catálogo.
 Retorna 200 si la conexión es correcta, o 503 si falla.
 """
 db_url = os.getenv('DATABASE_URL', 'postgres://cataloguser:catalogpass@catalog-db:5432/catalogdb')
 try:
 if db_url.startswith('postgres://') or db_url.startswith('postgresql://'):
 # Validación con psycopg2
 conn = psycopg2.connect(db_url, connect_timeout=3)
 with conn.cursor() as cursor:
 cursor.execute("SELECT 1;")
 conn.close()
 else:
 # Fallback local con sqlite3
 import sqlite3
 conn = sqlite3.connect(':memory:')
 cursor = conn.cursor()
 cursor.execute("SELECT 1;")
 conn.close()
 except:
 return Response({
 'status': 'healthy',
 'database': 'connected',
```



PASO 12:

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: app-config
 labels:
 app: microservices
data:
 DJANGO_DEBUG: "False"
 AUTH_SERVICE_URL: "http://auth-service:8000"
```

PASO 13:

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4 name: auth-deploy
5 labels:
6 app: auth
7 spec:
8 replicas: 2
9 selector:
10 matchLabels:
11 app: auth
12 template:
13 metadata:
14 labels:
15 app: auth
16 spec:
17 containers:
18 - name: auth
19 image: auth-service:latest
20 imagePullPolicy: IfNotPresent
21 ports:
22 - containerPort: 8000
23 env:
24 - name: DATABASE_URL
25 valueFrom:
26 secretKeyRef:
27 name: auth-db-secret
```

PASO 14:



```
1 apiVersion: v1
2 kind: Service
3 metadata:
4 name: auth-service
5 labels:
6 app: auth
7 spec:
8 type: ClusterIP
9 ports:
10 - port: 8000
11 targetPort: 8000
12 protocol: TCP
13 name: http
14 selector:
15 app: auth
16
```

PASO 15:

```
apiVersion: v1
kind: Service
metadata:
 name: catalog-service
 labels:
 app: catalog
spec:
 type: ClusterIP
 ports:
 - port: 8000
 targetPort: 8000
 protocol: TCP
 name: http
 selector:
 app: catalog
```



PASO 16:

```
Configuración de Base de Datos - Servicio de Autenticación
AUTH_DB_NAME=authdb
AUTH_DB_USER=authuser
AUTH_DB_PASS=authpass
AUTH_DB_HOST=auth-db
AUTH_DB_PORT=5432
AUTH_DATABASE_URL=postgres://authuser:authpass@auth-db:5432/authdb

Configuración de Base de Datos - Servicio de Catálogo
CATALOG_DB_NAME=catalogdb
CATALOG_DB_USER=cataloguser
CATALOG_DB_PASS=catalogpass
CATALOG_DB_HOST=catalog-db
CATALOG_DB_PORT=5432
CATALOG_DATABASE_URL=postgres://cataloguser:catalogpass@catalog-db:5432/catalogdb

Variables de Negocio e Inter-Servicio
JWT_SECRET=cambiar-esto-en-produccion-1234567890
AUTH_SERVICE_URL=http://auth-service:8000
```

PASO 17:

```
1. Verificar el Health de Auth Service
print("\n--- Test 1: Healthcheck de Auth Service (/health/) ---")
res_auth_health = requests.get("http://127.0.0.1:8001/health/")
print(f"Status Code: {res_auth_health.status_code}")
print(f"Response: {res_auth_health.json()}")

2. Verificar el Health de Catalog Service
print("\n--- Test 2: Healthcheck de Catalog Service (/health/) ---")
res_catalog_health = requests.get("http://127.0.0.1:8002/health/")
print(f"Status Code: {res_catalog_health.status_code}")
print(f"Response: {res_catalog_health.json()}")

3. Consumir catálogo sin cabecera de autenticación (401 Esperado)
print("\n--- Test 3: Consumo de Catálogo sin Token (401 Esperado) ---")
res_cat_no_token = requests.get("http://127.0.0.1:8002/catalog/")
print(f"Status Code: {res_cat_no_token.status_code}")
print(f"Response: {res_cat_no_token.json()}")

4. Obtener token JWT de auth-service
print("\n--- Test 4: Obtención de Token JWT (/login/) ---")
res_login = requests.post("http://127.0.0.1:8001/login/")
print(f"Status Code: {res_login.status_code}")
token_data = res_login.json()
print(f"Response: {token_data}")
token = token_data.get("token")
```

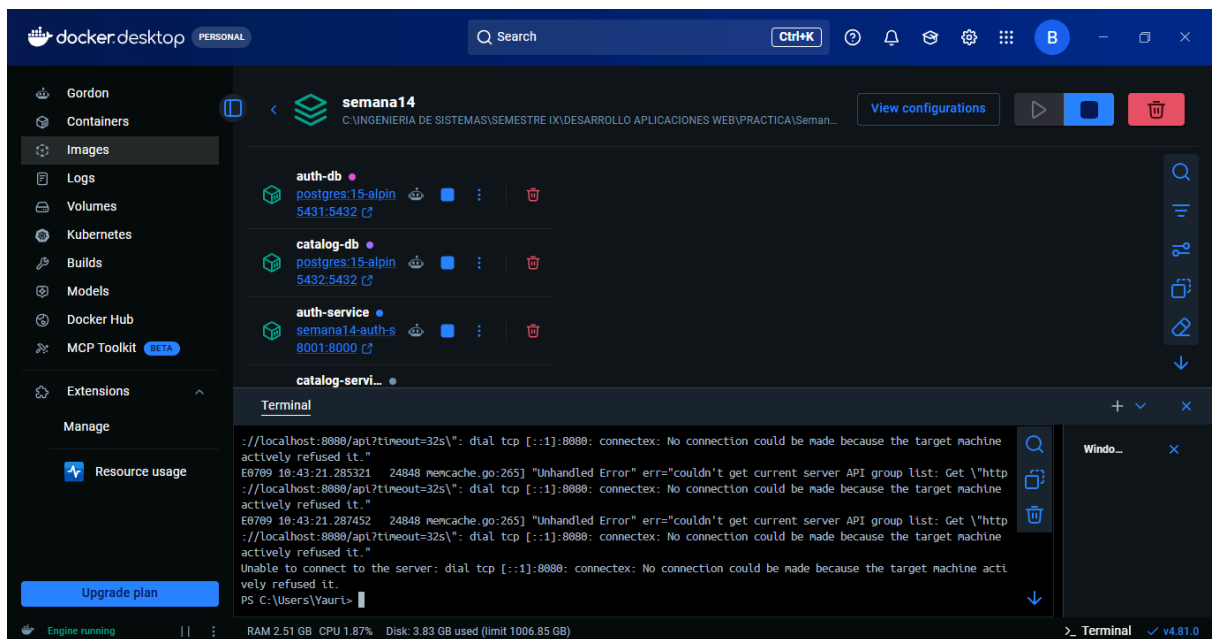


# UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ FACULTAD DE INGENIERÍA DE SISTEMAS



## Visualización con docker

```
PS C:\INGENIERIA DE SISTEMAS\SEMESTRE IX\DESARROLLO APLICACIONES WEB\PRACTICA\Semana 14\semana14> docker-compose up --build
catalog-service | [2026-07-09 15:53:51 +0000] [7] [INFO] Booting worker with pid: 7
catalog-service | [2026-07-09 15:53:51 +0000] [8] [INFO] Booting worker with pid: 8
catalog-service | [2026-07-09 15:53:51 +0000] [1] [INFO] Control socket listening at /home/appuser/.gunicorn/gunicorn.ctl
auth-service | Not Found: /static/rest_framework/css/prettify.css
auth-service | Not Found: /static/rest_framework/css/bootstrap-tweaks.css
auth-service | Not Found: /static/rest_framework/css/default.css
auth-service | Not Found: /static/rest_framework/js/ajax-form.js
auth-service | Not Found: /static/rest_framework/js/jquery-3.7.1.min.js
auth-service | Not Found: /static/rest_framework/js/csrf.js
auth-service | Not Found: /static/rest_framework/js/bootstrap.min.js
auth-service | Not Found: /static/rest_framework/js/prettify-min.js
auth-service | Not Found: /static/rest_framework/js/default.js
auth-service | Not Found: /static/rest_framework/js/load-ajax-form.js
auth-service | Not Found: /static/rest_framework/css/bootstrap.min.css
auth-service | Not Found: /static/rest_framework/css/bootstrap.min.css
auth-service | Not Found: /static/rest_framework/css/bootstrap-tweaks.css
```





http://localhost:8001/health/

[Django REST framework](#)

- [Health](#)

[GET](#)

- [json](#)
- [api](#)

OPTIONS

## Health

Endpoint '/health/' (GET)

Realiza una validación real de la conexión a la base de datos PostgreSQL.  
Retorna 200 si la conexión es exitosa, o 503 si falla.

GET /health/

HTTP 200 OK

Allow: OPTIONS, GET

Content-Type: application/json

Vary: Accept

```
{
 "status": "healthy",
 "database": "connected",
 "service": "auth-service"
}
```

http://localhost:8002/health/

[Django REST framework](#)

- [Health](#)

[GET](#)

- [json](#)
- [api](#)

OPTIONS

## Health

Endpoint '/health/' (GET)

Valida la conexión real a PostgreSQL del servicio de catálogo.  
Retorna 200 si la conexión es correcta, o 503 si falla.

GET /health/

HTTP 200 OK

Allow: OPTIONS, GET

Content-Type: application/json

Vary: Accept

```
{
 "status": "healthy",
 "database": "connected",
 "service": "catalog-service"
}
```





# UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ FACULTAD DE INGENIERÍA DE SISTEMAS



Postman interface showing a successful POST request to `http://localhost:8001/login/`.

**Request Details:**

- Method: POST
- URL: `http://localhost:8001/login/`

**Response Details:**

- Status: 200 OK
- Time: 542 ms
- Size: 486 B

**Response Body (JSON):**

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImVzdHVkaWVudGVfc2VtYW5hMTQ1LCJleHAiOjE3ODM2Mjc1MTd9.~AwEyGh8dM~-L87tBwgj5L9s3MQBibboiCyZp_C6Cx0"}
```